# Homework — 2.3.1 Algorithms — ANSWER SHEET

*Separate answer sheet / mark scheme — keep for marking.*

## Mark scheme

**Part A (14 marks)**

**1.** - (a) **[1]** — **O(log n)** (binary search). - (b) **[1]** — Each step **halves** the remaining search space, so the number of steps grows with **$\log_2$ n** (doubling n adds only one extra step) → O(log n); **precondition — the list must be sorted** (both the justification idea and the "sorted" precondition needed for the mark).

**2. [3]** — Bubble sort (largest value bubbles right each pass). 1 mark per correct pass row for any three of Passes 1–4 (max 3). Note Pass 4 still performs a swap (2 ↔ 1) — the list is only fully sorted after Pass 4.

Start: `[6, 2, 9, 4, 1]`

| After pass | Result |
|---|---|
| Pass 1 | `[2, 6, 4, 1, 9]` |
| Pass 2 | `[2, 4, 1, 6, 9]` |
| Pass 3 | `[2, 1, 4, 6, 9]` |
| Pass 4 | `[1, 2, 4, 6, 9]` |

Working — Pass 1: 6,2→swap `[2,6,9,4,1]`; 6,9→no; 9,4→swap `[2,6,4,9,1]`; 9,1→swap `[2,6,4,1,9]`. Pass 2: 2,6→no; 6,4→swap `[2,4,6,1,9]`; 6,1→swap `[2,4,1,6,9]`; 6,9→no. Pass 3: 2,4→no; 4,1→swap `[2,1,4,6,9]`; 4,6→no; 6,9→no. Pass 4: 2,1→swap `[1,2,4,6,9]`; 2,4→no; 4,6→no; 6,9→no.

**3. [3]** — Binary search for **31** in `[4, 9, 15, 18, 23, 27, 31, 42, 56]`. mid = (low + high) DIV 2.

| Step | low | high | mid (index) | value at mid | vs target 31 |
|---|---|---|---|---|---|
| 1 | 0 | 8 | 4 | 23 | 23 < 31 → set low = mid + 1 = 5 |
| 2 | 5 | 8 | 6 | 31 | = → **found** |

- Step 1: mid = (0+8) DIV 2 = 4; value 23 < 31, discard lower half → low = 5 (1 mark).
- Step 2: mid = (5+8) DIV 2 = 6; value 31 = target (1 mark).
- Result: **31 found at index 6** (1 mark).

**4. [3]** — Tree: root 55; left subtree 32 (children 20, 45); right subtree 78 (children 60, 90). - (a) **In-order** (Left → Node → Right): **20, 32, 45, 55, 60, 78, 90** *(1)* — note this is ascending, as expected for a BST. - (b) **Pre-order** (Node → Left → Right): **55, 32, 20, 45, 78, 60, 90** *(1)* - (c) **Post-order** (Left → Right → Node): **20, 45, 32, 60, 90, 78, 55** *(1)*

**5. [3]** — Dijkstra from A. Final shortest distances:

| Node | Shortest distance from A | Via |
|---|---|---|
| A | 0 | — |
| B | 2 | A |
| C | 3 | B |
| D | 6 | C |
| E | 8 | D |

Working: - A = 0. Tentative from A: B = 2, C = 5. Settle **B = 2 (via A)** (smallest). - From B: C via B = 2 + 1 = **3** (< 5, update **C = 3 via B**); D via B = 2 + 7 = 9. Settle **C = 3 (via B)**. - From C: D via C = 3 + 3 = **6** (< 9, update **D = 6 via C**); E via C = 3 + 8 = 11. Settle **D = 6 (via C)**. - From D: E via D = 6 + 2 = **8** (< 11, update **E = 8 via D**). Settle **E = 8 (via D)**.

Award up to 3 marks: all four distances correct (B=2, C=3, D=6, E=8) = 3; two or three correct = 2; one correct = 1; "via" entries support the working. Note C and D are updated from their initial tentative values (5 and 9) — full credit only for the final settled distances.

**Part B (6 marks)**

**6. [2]** — 1 mark: **merge sort** guarantees **O(n log n)** in the worst case. 1 mark: **quick sort**'s worst case is **O(n²)** (poor pivot / already-sorted data).

**7. [2]** — 1 mark: a **queue** (FIFO). 1 mark: a queue processes nodes in the order they are discovered, so the tree/graph is explored **level by level** (breadth-first); the first-in node is the first explored.

**8. [2]** — 1 mark: **intractable**. 1 mark: valid example — brute-force Travelling Salesman Problem; Boolean satisfiability by brute force; brute-force password cracking (any genuinely exponential-or-worse problem).

**Part C (5 marks)**

**9. [5]** — Indicative content; reward valid comparison points and a justified recommendation. Top band (4–5) for a developed comparison that explicitly justifies the choice using the **all-destinations** nature of the task.

- Both find the **optimal (shortest) path** when all edge weights are **non-negative** (A *also requires an admissible heuristic)* (1)*.
- **Dijkstra** uses only the actual cost so far g(n) and expands **outward in all directions**, naturally producing the shortest path to **every** node from the source *(1)*.
- **A*** uses **f(n) = g(n) + h(n)**, a heuristic estimate of cost to **a single goal**, which directs the search toward **one** target — it is not designed to find all shortest paths at once *(1)*.
- Because the task needs the shortest distance to **every** delivery point (many destinations), A *would have to be re-run per target and gains little, whereas Dijkstra solves all of them in a single run* (1)*.
- **Recommendation: Dijkstra's algorithm**, as the single-source-to-all-nodes requirement matches exactly what Dijkstra computes; A *would be preferable only if a single known target were needed and a good heuristic were available* (1)*.

---

**Total: 25 marks** (Part A = 2 + 3 + 3 + 3 + 3 = 14, Part B = 6, Part C = 5).